Soapies Platform — Technical Architecture Breakdown

Author: Manus AI

Date: March 24, 2026

Version: 55ebe5bc

Codebase: ~21,600 lines of TypeScript across 100+ files

1. Executive Summary

Soapies is a full-stack community membership platform built as a single-page application (SPA) with a monorepo structure. The platform manages member applications, event ticketing, social interactions, messaging, and administrative operations. The architecture follows a **type-safe end-to-end** pattern where TypeScript types flow from the database schema through the API layer to the React frontend, eliminating an entire class of runtime errors. The system is designed as mobile-first with premium glassmorphism UI, animated transitions, and a pink/purple brand identity derived from the Soapies logo.

2. High-Level Architecture

The platform follows a three-tier architecture with clear separation of concerns between the presentation layer, application layer, and data layer. All communication between the frontend and backend occurs through a single tRPC endpoint (`/api/trpc`), with the exception of a dedicated file upload route (`/api/upload/photo`).

Layer	Technology	Responsibility
Presentation	React 19, Tailwind CSS 4, Framer Motion, shadcn/ui	UI rendering, animations, routing, state management
Application	Express 4, tRPC 11, Drizzle ORM	Business logic, authentication, API procedures, notifications
Data	MySQL/TiDB (56 tables), AWS S3	Persistent storage, file storage

External	Manus OAuth, SendGrid, Twilio	Authentication, email, SMS

---

3. Frontend Architecture

3.1 Technology Stack

The frontend is built with **React 19** using **Vite 7** as the build tool and dev server. Styling is handled through **Tailwind CSS 4** with OKLCH color tokens defined in `client/src/index.css`. The component library is **shadcn/ui** (50+ pre-built components) extended with **Framer Motion** for page transitions, micro-interactions, and scroll-triggered animations. Routing uses **wouter** (a lightweight 1.5KB alternative to React Router) for SPA navigation.

3.2 File Structure

Plain Text

```
client/
├── src/
│   ├── _core/hooks/useAuth.ts       ← Auth state hook (Manus OAuth)
│   ├── components/
│   │   ├── Navbar.tsx               ← Glassmorphic nav with notification bell
│   │   ├── PageWrapper.tsx          ← Animated page transitions + floating pa
│   │   ├── FloatingElements.tsx     ← Ambient bubble particle system
│   │   ├── RequireProfile.tsx       ← Route guard for profile completion
│   │   └── ui/                      ← 50+ shadcn/ui components
│   ├── hooks/
│   │   ├── useProfileStatus.ts      ← Profile completion check
│   │   └── useMobile.tsx            ← Responsive breakpoint detection
│   ├── pages/
│   │   ├── Home.tsx                 ← Landing page (30,734 LOC)
│   │   ├── Events.tsx               ← Event listing with countdown (28,878 LO
│   │   ├── EventDetail.tsx          ← Ticket reservation form (34,628 LOC)
│   │   ├── Apply.tsx                ← 5-step application form (42,785 LOC)
│   │   ├── Dashboard.tsx            ← Member dashboard (23,412 LOC)
│   │   ├── Profile.tsx              ← 4-tab profile management (40,787 LOC)
│   │   ├── Wall.tsx                 ← Community social feed (18,098 LOC)
│   │   ├── Messages.tsx             ← Chat/messaging UI (15,956 LOC)
│   │   └── admin/                   ← 6 admin pages (AdminLayout, Dashboard, 
│   ├── lib/trpc.ts                  ← tRPC client binding
│   ├── App.tsx                      ← Route definitions + layout
│   ├── main.tsx                     ← tRPC/QueryClient providers
```

```
│   └── index.css                    ← Theme tokens (OKLCH), glassmorphism uti
└── index.html                       ← Google Fonts (Fredoka + DM Sans)
```

3.3 Routing Architecture

The application defines **14 routes** with two distinct access patterns. Public routes (Home, Events, EventDetail) are accessible without authentication. Protected routes (Dashboard, Profile, Wall, Messages) are wrapped in a `RequireProfile` guard component that checks both authentication status and profile completion, redirecting incomplete users to `/apply`. Admin routes use a role-based check (`ctx.user.role === 'admin'`) enforced at both the frontend (conditional rendering) and backend (adminProcedure middleware).

Route	Component	Access	Guard
`/`	Home	Public	None
`/events`	Events	Public	None
`/events/:id`	EventDetail	Public	None
`/apply`	Apply	Authenticated	Auth only
`/dashboard`	Dashboard	Authenticated	RequireProfile
`/profile`	Profile	Authenticated	RequireProfile
`/wall`	Wall	Authenticated	RequireProfile
`/messages`	Messages	Authenticated	RequireProfile
`/admin`	AdminDashboard	Admin	Role check
`/admin/events`	AdminEvents	Admin	Role check
`/admin/users`	AdminUsers	Admin	Role check
`/admin/applications`	AdminApplications	Admin	Role check
`/admin/settings`	AdminSettings	Admin	Role check
`/404`	NotFound	Public	None

3.4 State Management

The application uses **TanStack Query** (via tRPC React hooks) as the primary state management solution. There is no Redux, Zustand, or other global state library. Authentication state is provided by the `useAuth()` hook which calls `trpc.auth.me.useQuery()`. All server state is managed through tRPC query/mutation hooks with proper `staleTime`, `refetchOnWindowFocus`, and `enabled` configurations to prevent unnecessary re-fetching.

Mutations use **optimistic updates** for instant feedback on list operations (likes, comments, mark-read) and `invalidate` patterns for critical operations (reservations, profile updates, application submission).

3.5 Design System

The visual identity is built around a pink/purple color palette extracted from the Soapies logo. The theme is defined using CSS custom properties in OKLCH color space within `index.css`, providing consistent color tokens across all components.

Token	Purpose	Value
`--primary`	Buttons, CTAs, active states	Hot pink (`oklch(0.65 0.28 340)`)
`--secondary`	Supporting elements	Soft lavender
`--accent`	Highlights, badges	Deep purple
`--background`	Page backgrounds	Near-white with pink tint
`--card`	Card surfaces	White with glassmorphism

The design system includes custom utility classes for glassmorphism (`glass-card`, `glass-panel`), animated gradients (`gradient-shift`), and floating particle backgrounds. Typography uses **Fredoka** for display headings and **DM Sans** for body text, loaded via Google Fonts CDN.

---

4. Backend Architecture

4.1 Technology Stack

The backend runs on **Express 4** with **tRPC 11** providing the RPC layer. Database access is through **Drizzle ORM** connected to a **MySQL/TiDB** instance. Authentication uses **Manus OAuth** with JWT session cookies signed via the **jose** library. Data serialization between client and server uses **superjson**, which preserves JavaScript types like `Date` objects across the wire.

4.2 Server Entry Point

The server boots from `server/_core/index.ts`, which initializes Express, mounts the OAuth callback route (`/api/oauth/callback`), the tRPC middleware (`/api/trpc`), the photo upload route (`/api/upload/photo`), and the Vite dev server middleware for serving the frontend in development.

4.3 tRPC Router Architecture

The API is organized into **13 feature routers** containing **60+ procedures** (queries and mutations). Each router is a self-contained module handling a specific domain concern. Procedures are classified as either `publicProcedure` (no auth required) or `protectedProcedure` (requires valid session). Admin procedures use an additional middleware that checks `ctx.user.role === 'admin'`.

Plain Text

```
appRouter
├── system          ← Health check, owner notifications
├── auth            ← me (session check), logout
├── profile         ← me, upsert, submitApplication, photos, uploadPhoto, delet
├── events          ← list, byId, addons, feedback, submitFeedback
├── reservations    ← myReservations, create
├── wall            ← posts, create, like, comments, addComment, myLikes
├── messages        ← conversations, messages, send, createConversation
├── applicationTimeline ← logs (application status history)
├── notifications   ← list, unreadCount, markRead, markAllRead, channels, prefe
├── referrals       ← myCode, generate
├── credits         ← balance, history
├── admin           ← stats, users, profiles, pendingApplications, reviewApplic
│                     events (CRUD), reservations, auditLogs, settings, updateS
└── communities     ← list, bySlug
```

4.4 Database Helpers (`server/db.ts`)

The `db.ts` file contains **62 exported async functions** that encapsulate all database queries. These helpers return raw Drizzle result objects and are consumed by tRPC procedures. This separation ensures that database logic is reusable across multiple procedures and testable in isolation.

Key helper categories include user management (upsert, lookup by openId/id), profile CRUD (create, update, status changes), event queries (list with filters, by ID with pricing), reservation management, wall post operations, messaging queries, notification CRUD, credit balance tracking, and admin aggregate queries.

4.5 Authentication Flow

Authentication follows the **OAuth 2.0 Authorization Code** flow with Manus as the identity provider.

1. The frontend calls `getLoginUrl()` which constructs a Manus OAuth URL with the app's origin encoded in the `state` parameter.
2. The user authenticates on the Manus login portal.
3. Manus redirects to `/api/oauth/callback` with an authorization code.
4. The server exchanges the code for user information, upserts the user record in the database, signs a JWT session token, and sets it as an HTTP-only cookie.
5. Subsequent requests include the cookie automatically. The tRPC context builder (`server/_core/context.ts`) verifies the JWT and injects `ctx.user` for every request.
6. `protectedProcedure` middleware checks for `ctx.user` and throws `UNAUTHORIZED` if missing.
7. `adminProcedure` additionally checks `ctx.user.role === 'admin'` .

The owner's account (identified by `OWNER_OPEN_ID` environment variable) is automatically promoted to admin role on first login.

---

5. Database Architecture

5.1 Overview

The database layer uses **MySQL/TiDB** accessed through **Drizzle ORM** with a schema-first workflow. The schema is defined in `drizzle/schema.ts` and migrations are managed via `drizzle-kit generate && drizzle-kit migrate` (aliased as `pnpm db:push`). The database contains **56 tables** organized into 8 functional domains.

5.2 Table Organization

Domain	Tables	Key Tables
User & Identity	3	`users` , `profiles` , `userGroups`
Application Pipeline	2	`applicationPhotos` , `applicationLogs`
Relationships	3	`relationshipGroups` , `relationshipGroupMembers` , `partnerInvitations`

Events & Ticketing	11	`events`, `reservations`, `tickets`, `eventAddons`, `reservationAddons`, `eventPromotionalPricing`, `eventOperators`, `eventShifts`, `shiftAssignments`, `eventSetupChecklist`, `eventFeedback`
Messaging	8	`conversations`, `conversationParticipants`, `messages`, `messageReactions`, `messageReads`, `pinnedMessages`, `typingIndicators`, `userPresence`
Community/Social	4	`wallPosts`, `wallPostLikes`, `wallPostComments`, `announcements`
Notifications	8	`notifications`, `notificationQueue`, `notificationBatches`, `notificationPreferences`, `adminNotificationPreferences`, `notificationDeliveryRules`, `notificationAnalytics`, `pushSubscriptions`, `smsNotifications`
Admin & Operations	17	`adminAuditLogs`, `appSettings`, `groups`, `preApprovedPhones`, `singleMaleInviteCodes`, `introCallSlots`, `profileChangeRequests`, `groupChangeRequests`, `signedWaivers`, `memberCredits`, `referralCodes`, `expenses`, `resourceAcknowledgments`, `cancellationRequests`, `blockedUsers`

5.3 Key Schema Patterns

The schema follows several consistent patterns. All tables use auto-incrementing integer primary keys. Timestamps are stored as UTC `timestamp` columns with `defaultNow()`. Foreign keys reference the `users.id` column for user ownership. Enum columns use `mysqlEnum` for type safety (e.g., profile status: `draft | pending | approved | rejected | waitlisted`). Decimal columns are used for monetary values (`decimal(10,2)`). The `profiles` table serves as the central entity for member data, linked 1:1 with `users` via `userId`.

5.4 Event Data Model

Events are the core business entity with a rich data model supporting three event types (`house_party` , `rave` , `beach_party`), three ticket pricing tiers (`priceSingleFemale` , `priceSingleMale` , `priceCouple`), capacity management, and operational logistics (operators, shifts, setup checklists). The database is pre-seeded with **39 real events** spanning April 2025 through March 2026.

---

6. Notification System

6.1 Architecture

The notification service (`server/notifications.ts`) implements a **unified multi-channel notification system** with three delivery channels: in-app, email (SendGrid), and SMS (Twilio). The service follows a graceful degradation pattern where email and SMS channels are automatically disabled if their respective API keys are not configured, while in-app notifications always work.

6.2 Channel Configuration

Channel	Provider	Status	Activation
In-App	Built-in (database)	Always active	Automatic
Email	SendGrid	Conditional	Provide `SENDGRID_API_KEY` + `SENDGRID_FROM_EMAIL`

SMS	Twilio	Conditional	Provide `TWILIO_ACCOUNT_SID` + `TWILIO_AUTH_TOKEN` + `TWILIO_PHONE_NUMBER`

6.3 Notification Flow

When an admin reviews an application, the `reviewApplication` procedure calls `sendNotification()` with a template generated by `buildApprovalNotification()`, `buildRejectionNotification()`, or `buildWaitlistNotification()`. The service then dispatches across all enabled channels simultaneously. Each template includes channel-specific content: a title/body for in-app, an HTML email with subject line for SendGrid, and a concise text message for Twilio.

6.4 User Preferences

Users can configure notification preferences per category (e.g., events, applications, messages) with independent toggles for each channel (in-app, email, SMS, push). Preferences are stored in the `notificationPreferences` table and checked before dispatching.

---

7. File Storage

File storage uses **AWS S3** via the `@aws-sdk/client-s3` library. The `storagePut()` helper uploads file bytes to S3 and returns a public CDN URL. The `storageGet()` helper generates presigned URLs for private access. Photo uploads flow through a dedicated Express route (`/api/upload/photo`) that accepts multipart form data, validates file type and size, and stores the file in S3 with a randomized key to prevent enumeration.

Static assets (logo, hero images, event artwork) are pre-uploaded to CDN and referenced by URL throughout the application. No static media files are stored in the project directory to prevent deployment timeouts.

---

8. Testing Architecture

The test suite uses **Vitest** with **60 tests across 3 test files**. Tests are structured as integration tests that call tRPC procedures through the `appRouter.createCaller()` API, testing

the full procedure chain (input validation, auth middleware, database queries) without HTTP overhead.

Test File	Tests	Coverage
`server/auth.logout.test.ts`	1	Session cookie clearing
`server/notifications.test.ts`	15	Notification CRUD, channels, preferences, templates
`server/soapies.test.ts`	44	All feature routers: events, profiles, reservations, wall, messages, credits, referrals, admin, application flow

Tests use three context factories (`createPublicContext` , `createUserContext` , `createAdminContext`) to simulate different authentication states. Each protected procedure is tested for both unauthorized rejection and authorized access.

---

9. Application Onboarding Flow

The member application process is a **5-step wizard** implemented in `Apply.tsx` (42,785 bytes, the largest page component). The flow is designed to collect comprehensive profile information while maintaining engagement through animated step transitions and progress indicators.

Step	Title	Data Collected
1	Welcome & Basics	Display name, gender, date of birth, phone number
2	About You	Bio, sexual orientation, relationship status, location, community selection
3	Photos	1-6 profile photos uploaded to S3
4	Preferences	Interests (multi-select), what you're looking for
5	Review & Submit	Summary review, community guidelines acceptance, liability waiver

Each step auto-saves progress via `profile.upsert` mutation. The `RequireProfile` route guard redirects authenticated users without a completed profile to `/apply`, ensuring all members complete onboarding before accessing protected features.

---

10. Environment Variables

The platform uses a structured environment variable system managed through the Manus platform. Variables are injected at runtime and accessed via `server/_core/env.ts`.

Variable	Purpose	Layer
`DATABASE_URL`	MySQL/TiDB connection string	Server
`JWT_SECRET`	Session cookie signing key	Server
`VITE_APP_ID`	Manus OAuth application ID	Client
`OAUTH_SERVER_URL`	Manus OAuth backend URL	Server
`VITE_OAUTH_PORTAL_URL`	Manus login portal URL	Client
`OWNER_OPEN_ID`	Owner's Manus identifier (auto-admin)	Server
`BUILT_IN_FORGE_API_URL`	Manus built-in APIs (LLM, storage)	Server
`BUILT_IN_FORGE_API_KEY`	Bearer token for built-in APIs	Server
`SENDGRID_API_KEY`	SendGrid email API key (optional)	Server
`SENDGRID_FROM_EMAIL`	Sender email address (optional)	Server
`TWILIO_ACCOUNT_SID`	Twilio account SID (optional)	Server
`TWILIO_AUTH_TOKEN`	Twilio auth token (optional)	Server
`TWILIO_PHONE_NUMBER`	Twilio sender phone (optional)	Server

---

11. Build & Deployment

11.1 Development

The development server runs via `pnpm dev` which starts `tsx watch server/_core/index.ts`. This boots Express with Vite middleware for hot module replacement (HMR). The frontend and backend share a single port (3000) with the Vite dev server proxying API requests.

11.2 Production Build

The production build (`pnpm build`) executes two steps: `vite build` compiles the React frontend into static assets in `dist/`, and `esbuild` bundles the server code into `dist/index.js`. The production server (`pnpm start`) serves the static frontend and handles API requests on the same port.

11.3 Database Migrations

Schema changes follow a generate-then-migrate workflow: edit `drizzle/schema.ts`, then run `pnpm db:push` which executes `drizzle-kit generate` (creates SQL migration files in `drizzle/migrations/`) followed by `drizzle-kit migrate` (applies pending migrations to the database).

---

12. Security Considerations

The platform implements several security measures. Authentication tokens are stored in **HTTP-only, Secure, SameSite=None** cookies to prevent XSS access. All API mutations validate input through **Zod schemas** at the tRPC layer. Admin operations are gated by a `role` check in the `adminProcedure` middleware. File uploads validate content type and size before S3 storage. S3 file keys include random suffixes to prevent enumeration attacks. The `OWNER_OPEN_ID` environment variable ensures the platform owner always has admin access.

---

13. Performance Optimizations

The frontend implements several performance patterns. TanStack Query provides automatic request deduplication, caching with configurable `staleTime`, and background refetching. Framer Motion animations use `layout` and `layoutId` props for GPU-accelerated transitions. The `useInView` hook from Framer Motion triggers animations only when elements enter the viewport, reducing initial render cost. Query `enabled` flags prevent

unnecessary API calls when data dependencies are not yet available. The `refetchOnWindowFocus: false` setting on stable data queries prevents excessive re-fetching.

---

14. Summary Statistics

Metric	Value
Total lines of TypeScript	~21,600
Frontend pages	14 (8 member + 6 admin)
Shared components	10 custom + 50+ shadcn/ui
Database tables	56
tRPC routers	13
tRPC procedures	60+
DB helper functions	62
Test cases	60
Test files	3
Seeded events	39
Notification channels	3 (in-app, email, SMS)
External integrations	4 (Manus OAuth, S3, SendGrid, Twilio)